Analysis and Design of Algorithms (ADA)

Lab Assignment – 1

Task :

Understanding Algorithm Efficiency through Analysis, Recurrences, and Sorting

Submitted by : Md. Anas khan

Submitted to : Dr. Aarti

Course : B.Tech – CSE (Data Science)

Roll no. : 2401420035

Task 1: Algorithm Growth Observation

Aim: To observe how execution time changes for algorithms with different time complexities.

Input Sizes Used: n = 10, 50, 100, 200, 400, 800, 1600

Code

```
import time
import math
import matplotlib.pyplot as plt

sizes =[10,50,100,200,400,800,1600]

constant_times =[]
linear_times =[]
quadratic_times =[]
log_times =[]

def constant():
    x = 5 +5
def linear(n):
    for i in range(n):
        x =i
def quadratic(n):
    for i in range(n):
        for j in range(n):
            x = i +j
def logarithmic(n):
    i =1
    while i < n:
        i *= 2
for n in sizes:

    start =time.time()
    constant()
    constant_times.append(time.time() - start)

    start = time.time()
    linear(n)
    linear_times.append(time.time() - start)

    start = time.time()
    quadratic(n)
    quadratic_times.append(time.time() - start)
```

```
    start = time.time()
    logarithmic(n)
    log_times.append(time.time() - start)

plt.plot(sizes,constant_times, label="O(1)")
plt.plot(sizes,linear_times, label="O(n)")
plt.plot(sizes,quadratic_times, label="O(n^2)")
plt.plot(sizes,log_times, label="O(log n)")

plt.xlabel("Input Size")
plt.ylabel("Execution Time")
plt.title("Algorithm Growth Comparison")
plt.legend()
plt.show()
```

Observation

O(1) remains almost constant regardless of input size. O(log n) increases very slowly. O(n) increases linearly. O(n^2) grows very fast and dominates the execution time for large inputs.

Task 2: Best, Average, and Worst Case Analysis

Aim: To compare Linear Search and Binary Search execution time.

Code

```
import random

import time
import matplotlib.pyplot as plt

def linear_search(arr,target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr,target):
    low =0
    high = len(arr) -1

    while low <= high:
        mid = (low +high) //2
```

```
        if arr[mid] ==target:
            return mid
        elif arr[mid] <target:
            low =mid +1
        else:
            high =mid - 1
    return -1

sizes =[100,500,1000,5000,10000]
linear_times =[]
binary_times =[]

for n in sizes:
    arr = sorted(random.sample(range(1, n*10), n))
    target =arr[n//2]
    start = time.time()
    linear_search(arr,target)
    linear_times.append(time.time() - start)

    start = time.time()
    binary_search(arr,target)
    binary_times.append(time.time() - start)

plt.plot(sizes,linear_times,label="Linear Search")
plt.plot(sizes,binary_times,label="Binary Search")

plt.xlabel("Input Size")
plt.ylabel("Execution Time")
plt.title("Linear vs Binary Search")
plt.legend()
plt.show()
```

Observation

Linear search checks each element sequentially, so execution time increases with input size. Binary search repeatedly divides the array and therefore performs much faster for large datasets.

Task 3: Recursion and Recurrence Validation

Aim: To compare recursive algorithms with optimized approaches.

Code

```
import time
call_count =0
def factorial(n):
    if n ==0:
        return 1
    return n * factorial(n-1)

def fibonacci_recursive(n):
    global call_count
    call_count +=1
    if n <=1:
        return n
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)

def fibonacci_dp(n):
    dp = [0,1]
    for i in range(2,n+1):
        dp.append(dp[i-1] + dp[i-2])
    return dp[n]

n =10
start = time.time()
factorial(n)
print("Factorial Time:",time.time()-start)

call_count =0
start =time.time()
fibonacci_recursive(n)
print("Recursive Fibonacci Time:",time.time()-start)
print("Function Calls:",call_count)

start =time.time()
fibonacci_dp(n)
print("DP Fibonacci Time:",time.time()-start)
```

Observation

Recursive Fibonacci grows exponentially due to repeated calculations. Dynamic programming stores results and greatly improves performance.

Task 4: Solving Recurrences Through Code

Aim: Implement recursive functions for given recurrence relations.

Code

```
import time

calls1 =0
calls2 =0
def recurrence1(n):
    global calls1
    calls1 +=1
    if n <=1:
        return 1
    return recurrence1(n//2) +n

def recurrence2(n):
    global calls2
    calls2 +=1
    if n <=1:
        return 1
    return recurrence2(n//2) +recurrence2(n//2) +n

n =64
start =time.time()
recurrence1(n)
print("Recurrence 1 Time:",time.time()-start)
print("Calls:",calls1)
start =time.time()
recurrence2(n)
print("Recurrence 2 Time:",time.time()-start)
print("Calls:", calls2)
```

Observation

T(n) = T(n/2) + n follows approximately O(n). T(n) = 2T(n/2) + n grows faster and follows O(n log n).